



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №8
«Шаблони «Composite», «Flyweight», «Interpreter»,
«Visitor»
Варіант - 4

Виконала:
Студентка групи ІА-22
Третьякова Ельвіра

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості.....	2
Реалізація не менше 3-х класів відповідно до обраної теми.....	6
Реалізація шаблону за обраною темою.....	7
<i>Діаграма класів для паттерну Flyweight.....</i>	<i>10</i>
<i>Проблема, яку допоміг вирішити шаблон Flyweight</i>	<i>11</i>
<i>Переваги застосування паттерну Flyweight</i>	<i>12</i>
<i>Посилання на GitHub</i>	<i>13</i>
Висновок	13

Тема: Шаблони «Composite», «Flyweight», «Interpreter», «Visitor».

Мета: ознайомитися з шаблонами проектування «Composite», «Flyweight», «Interpreter», «Visitor» та набуті практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи

Варіант – 4: Графічний редактор (проху, prototype, decorator, bridge, flyweight, SOA)
Графічний редактор повинен вміти створювати / редагувати растрові (або векторні на розсуд студента) зображення в 2-3 основних популярних форматах (bmp, png, jpg), мати панель інструментів для створення графічних примітивів, вибору кольорів, нанесення тексту, додавання найпростіших візуальних ефектів (ч/б растр, інфрачервоний растр, 2-3 на вибір учня), роботи з шарами.

Короткі теоретичні відомості

Шаблони роботи з БД при розробці корпоративних додатків:

Існує набір сучасних шаблонів, які використовуються для розробників широко-масштабних корпоративних додатків - з кількістю користувачів понад 100 і великою кількістю взаємопов'язаних програм. Такі додатки вимагають окремих підходів до організації доступу до даних, оскільки кількість даних, що зберігаються, класів і клієнтів, що звертаються, зростає.

Шаблон «Active record» (Активний запис)

Один об'єкт управляє і даними, і поведінкою. Більшість цих даних постійні і їх треба зберігати в БД. Цей патерн використовує найбільш очевидний підхід - зберігання логіки доступу до даних в об'єкті сутності.

Об'єкт є «обгорткою» одного рядка з БД або подання, включає в себе доступ до БД і логіку поводження з даними. Шаблон використовується зокрема в Ruby On Rails додатках, з огляду на його простоту.

Однак зі збільшенням кількості різноманітних запитів зростає складність класу даних і як правило логіка запитів виноситься в окремий об'єкт.

Шаблон «Table Data Gateway» (Шлюз до даних)

В даному випадку окремий клас для кожного класу даних займається взаємодією з базою даних. Він зберігає в собі логіку всіх запитів (включаючи на збереження і видалення записів). Такий підхід більш гнучкий і тестований, і дозволяє утримувати окремо дані і окремо взаємодію з базою даних. Однак для кожного шлюзу до даних код взаємодії як правило повторюється (з'єднання з базою, формування команди джерела даних і т.п.), тому їх часто абстрагують в окремий базовий клас для шлюзів. Ще такий шаблон часто називають репозиторій.

Шаблон «Data mapping» (Відображення даних)

Вкрай часто виникають проблеми перетворення об'єкта даних в рядок реляційного джерела. Така ж проблема може виникати при необхідності передачі об'єкта по мережі. Для її вирішення прийнято створювати окремі об'єкти (або методи усередині класу даних) для перетворення об'єктів даних в дані, що приймаються джерелом виду. Ця процедура носить назву відображення (mapping). Під час відображення виправляються всі невідповідності типів даних між об'єктом даних і джерелом даних. Типовий маппер - з об'єкта даних в таблицю реляційного джерела даних. Він несе знання про назви колонок в таблиці і відповідні властивості об'єкта даних, які відповідають цим колонкам.

Шаблон «Composite»

Структура:

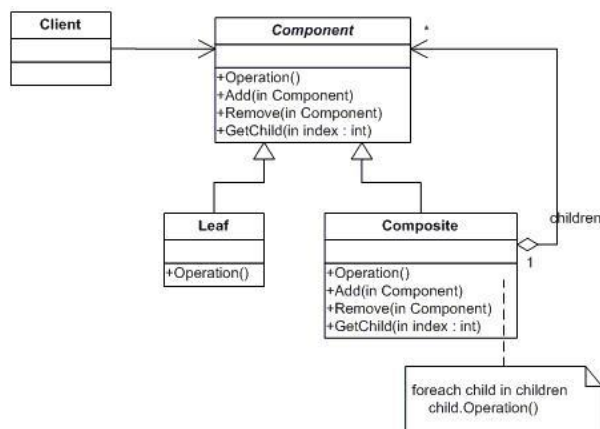


Рис.1 Структура шаблону Composite

Призначення:

Шаблон використовується для складання об'єктів в деревоподібну структуру для подання ієрархій типу «частина цілого». Даний шаблон дозволяє уніфіковано обробляти як поодинокі об'єкти, так і об'єкти з вкладеністю.

Простим прикладом може служити складання компонентів всередині звичайної форми.

Форма може містити дочірні елементи (поля для введення тексту, цифр, написи, малюнки тощо); дочірні елементи можуть в свою чергу містити інші елементи. Наприклад, при виконанні операції розтягування форми необхідно, щоб вся ієрархія розтягнулася відповідним чином. В такому випадку форма розглядається як композитний об'єкт і операція розтягування застосовується до всіх дочірніх елементів рекурсивно.

Даний шаблон зручно використовувати при необхідності подання та обробки ієрархій об'єктів.

Переваги та недоліки:

- + Спрощує архітектуру клієнта при роботі зі складним деревом компонентів.
- + Полегшує додавання нових видів компонентів.
- Створює занадто загальний дизайн класів.

Шаблон «Flyweight»

Структура:

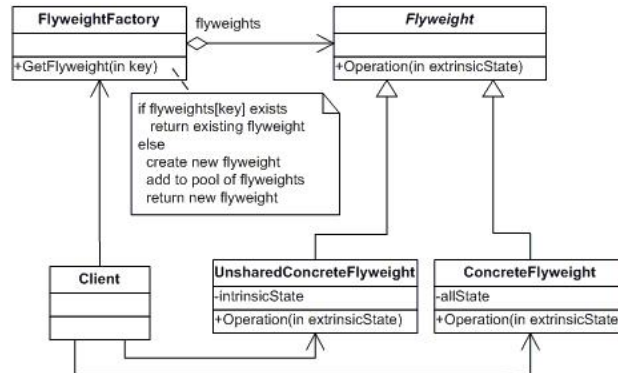


Рис.2 Структура шаблону Flyweight

Призначення:

Шаблон використовується для зменшення кількості об'єктів в додатку шляхом поділу цих об'єктів між ділянками додатку. Flyweight являє собою поділюваний об'єкт. Здається, ніби для кожної літери існує окремий об'єкт. Насправді фізично об'єкт всього один, існує лише безліч посилань на нього. Дуже важливою є концепція «внутрішнього» і «зовнішнього» станів. Внутрішній стан відображає дані, характерні саме поділюваному об'єкту (наприклад, код букви); зовнішній стан несе інформацію про його застосування в додатку (наприклад, рядок і стовпчик). Внутрішній стан зберігається в самому поділюваному об'єкті, зовнішній - в об'єктах додатку (контексту використання поділюваного об'єкта).

Даний шаблон дуже добре застосовувати у випадках, коли використовується безліч однакових об'єктів (наприклад, графічних примітивів).

Переваги та недоліки:

- + Заощаджує оперативну пам'ять.
- Витрачає процесорний час на пошук/обчислення контексту.
- Ускладнює код програми внаслідок введення безлічі додаткових класів.

Шаблон «Interpreter»

Структура:

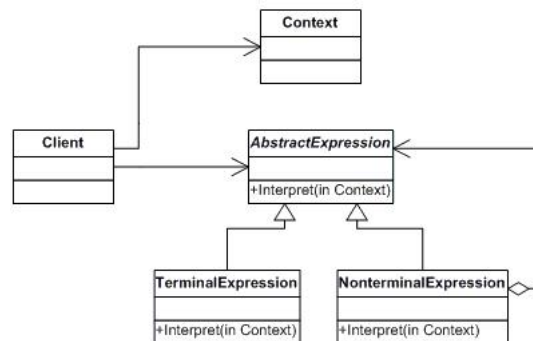


Рис.3 Структура шаблону Interpreter

Призначення:

Даний шаблон використовується для подання граматики і інтерпретатора для вибраної мови (наприклад, скриптової). Граматика мови представлена термінальними і нетермінальними символами, кожен з яких інтерпретується в контексті використання. Клієнт передає контекст і сформовану пропозицію в використовувану мову в термінах абстрактного синтаксичного дерева (деревоподібна структура, яка однозначно визначає ієрархію виклику підвиразів), кожен вираз інтерпретується окремо з використанням контексту. У разі наявності дочірніх виразів, батьківський вираз інтерпретує спочатку дочірні (рекурсивно), а потім обчислює результат власної операції.

Шаблон зручно використовувати в разі невеликої граматики (інакше розростеться кількість використовуваних класів) і відносно простого контексту (без взаємозалежностей і т.п.).

Даний шаблон визначає базовий каркас інтерпретатора, який за допомогою рекурсії повертає результат обчислення пропозиції на основі результатів окремих елементів.

При використанні даного шаблону дуже легко реалізовується і розширюється граматика, а також додаються нові способи інтерпретації виразів.

Переваги та недоліки:

+ Граматику стає легко розширювати та змінювати, реалізації класів, що описують вузли абстрактного синтаксичного дерева схожі (легко кодуються).

+ Можна легко змінювати спосіб обчислення виразів.

- Сопровождение грамматики с большим числом правил затруднительно.

Шаблон «Visitor»

Структура:

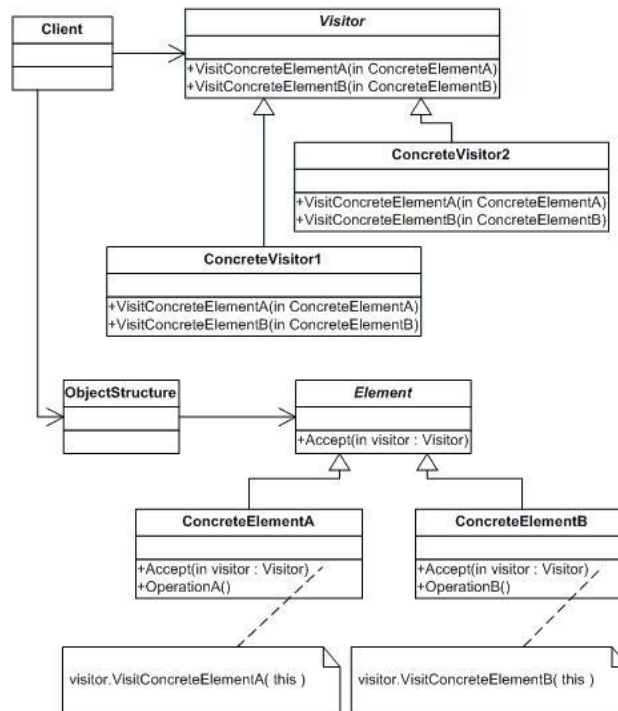


Рис.4 Структура шаблону Visitor

Призначення:

Шаблон відвідувач дозволяє вказувати операції над елементами без зміни структури конкретних елементів. Таким чином вкрай зручно додавати нові операції, проте дуже важко додавати нові елементи в ієрархію (необхідно додавати відповідні методи для обробки їх відвідувань в кожного відвідувача).

Даний шаблон дозволяє групувати однотипні операції, що застосовуються над різнотипними об'єктами.

Реалізація не менше 3-х класів відповідно до обраної теми

Структура проекту з реалізованими класами зображена на рисунку 5:

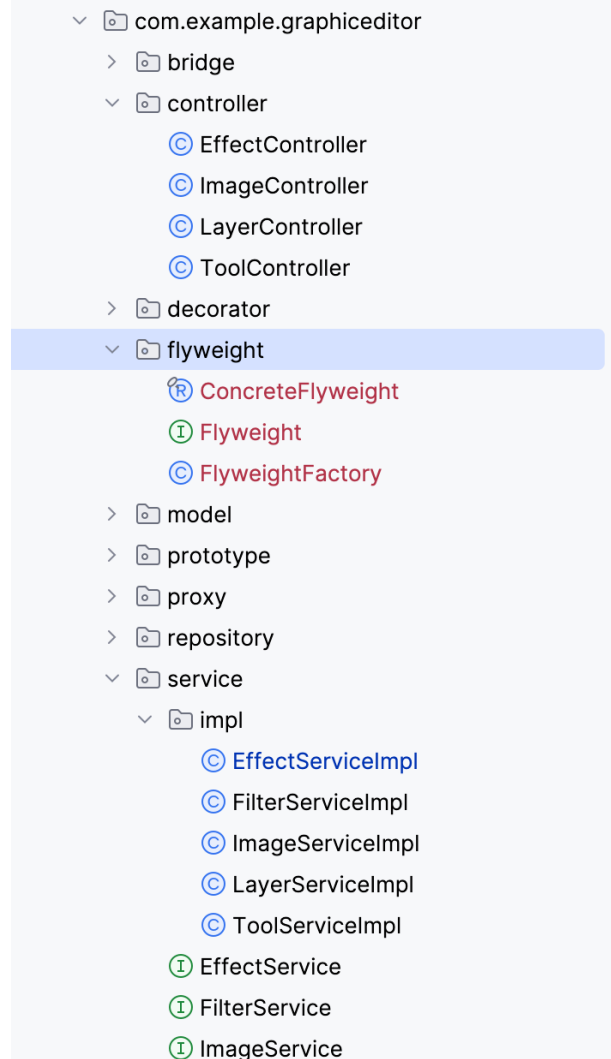


Рис.5 Структура проекту з реалізованими класами

Опис реалізації:

1. Базовий клас *Effect*:

Це абстракція для ефектів, які можуть бути застосовані до зображень. Клас визначає загальні методи для обробки ефектів, включаючи застосування та збереження.

2. Інтерфейс *Flyweight*:

Інтерфейс описує загальні операції для конкретних Flyweight-об'єктів, зокрема метод apply(), який буде реалізовано в конкретних класах Flyweight.

3. Конкретні реалізації *Flyweight*:

ConcreteFlyweight реалізує інтерфейс Flyweight і надає конкретну логіку для застосування ефекту, наприклад, для ефектів зміни яскравості, контрасту або застосування фільтрів.

4. Фабрика *FlyweightFactory*:

Це фабричний клас, який відповідає за створення та кешування об'єктів ConcreteFlyweight. Він забезпечує, щоб одна інстанція ефекту не створювалася кілька разів для різних зображень, тим самим економлячи пам'ять.

5. Інтерфейс EffectService та реалізація EffectServiceImpl:

EffectService надає методи для роботи з ефектами, включаючи створення, оновлення та застосування ефектів до зображень. Реалізація EffectServiceImpl використовує FlyweightFactory для отримання об'єктів Flyweight та застосування ефектів через методи класу Effect.

6. Контролер EffectController:

Контролер надає REST API для управління ефектами, включаючи створення, отримання, оновлення та застосування фільтрів до зображень. Він використовує сервіси для виклику відповідних методів у EffectService.

7. Абстракція та конкретні реалізації:

За допомогою патерну flyweight реалізовано відокремлення спільної частини ефектів (через FlyweightFactory) від конкретної логіки застосування ефектів (через ConcreteFlyweight).

Клас EffectServiceImpl виступає абстракцією для управління ефектами, а конкретні реалізації flyweight здійснюють точне застосування ефектів до зображень.

Використовуючи патерн Flyweight, система значно знижує використання пам'яті, оскільки одна інстанція ефекту може використовуватись для кількох зображень. Це забезпечує збереження ресурсів при застосуванні однотипних ефектів до різних зображень.

Реалізація патерну flyweight у системі дозволила значно знизити витрати пам'яті при застосуванні ефектів, одночасно забезпечивши гнучкість та можливість легко додавати нові типи ефектів без необхідності змінювати основну архітектуру коду. Це дозволяє системі бути масштабованою та ефективною навіть при великій кількості зображень, до яких застосовуються однакові або схожі ефекти.

Реалізація шаблону за обраною темою

У ході реалізації графічного редактора було застосовано патерн flyweight, який дозволяє зменшити споживання пам'яті завдяки використанню спільних об'єктів для повторюваних операцій, таких як застосування ефектів до зображень.

Основна мета використання цього патерна — оптимізація ресурсів шляхом уникнення дублювання даних для об'єктів, які мають однакову поведінку, але різні контексти.

Опис реалізації:

1. Інтерфейс Flyweight

Це основа для всіх класів, які представляють "легковагові" об'єкти. Він визначає загальний контракт для реалізації спільної поведінки.

Особливості:

- Визначає метод apply(String data), який приймає параметр data (контекст для застосування ефекту).
- Дає змогу забезпечити єдиний підхід до використання ефектів незалежно від їхнього типу.

```

1 package com.example.graphiceditor.flyweight;
2
3 public interface Flyweight { 5 usages 1 implementation
4     void apply(String data); 1 usage 1 implementation
5 }

```

Рис.6 Реалізація інтерфейсу Flyweight

2. Клас ConcreteFlyweight

Це реалізація інтерфейсу Flyweight. Кожен об'єкт цього класу відповідає конкретному ефекту (наприклад, "Чорно-білий", "Інфрачервоний растр").

Особливості:

- Використовує name для ідентифікації ефекту.
- Реалізує метод apply(String data):

Виконує операцію для контексту data.

Виводить повідомлення з іменем ефекту та контекстом, демонструючи, який ефект застосовується.

- Використання record забезпечує:

Ефективне управління пам'яттю (імутабельні об'єкти).

Автоматичне створення геттерів.

```

1 package com.example.graphiceditor.flyweight;
2
3 import lombok.Getter;
4
5 @Getter 1 usage
6 public record ConcreteFlyweight(String name) implements Flyweight {
7
8     @Override 1 usage
9     public void apply(String data) {
10         System.out.println("Applying effect: " + name + " with data: " + data);
11     }
12 }

```

Рис.7 Реалізація класу ConcreteFlyweight

3. Клас FlyweightFactory

Це фабрика, яка управляє створенням та зберіганням "легковагових" об'єктів. Використовує шаблон "Singleton" через Spring (позначення @Component).

Особливості:

Кешування:

- Зберігає створені об'єкти у mapі effects.
- Використовує computeIfAbsent() для перевірки наявності об'єкта:

Якщо об'єкт із заданим ім'ям вже існує, повертає його.

Інакше створює новий через конструктор ConcreteFlyweight та додає до мапи.

Метод getEffect(String name):

- Абстрагує створення або отримання об'єкта.
- Гарантує, що кожен ефект створюється лише один раз.

Метод getFlyweightCount():

- Повертає кількість унікальних "легковагових" об'єктів.

- Використовується для моніторингу оптимізації пам'яті.



```

1 package com.example.graphiceditor.flyweight;
2
3 import org.springframework.stereotype.Component;
4
5 import java.util.HashMap;
6 import java.util.Map;
7
8 @Component 3 usages
9 public class FlyweightFactory {
10     private final Map<String, Flyweight> effects = new HashMap<>(); 2 usages
11
12     public Flyweight getEffect(String name) { 1 usage
13         return effects.computeIfAbsent(name, ConcreteFlyweight::new);
14     }
15
16     public int getFlyweightCount() { 1 usage
17         return effects.size();
18     }
19 }

```

Рис.8 Реалізація класу FlyweightFactory

Загальний процес роботи

1. При запиті ефекту фабрика перевіряє наявність об'єкта в кеші.
2. Якщо об'єкта немає, він створюється і додається до кешу.
3. Усі подальші запити повертають існуючий об'єкт.
4. Метод apply() виконує операцію з переданими даними, використовуючи ефект.

Результати реалізації

Оптимізація пам'яті: об'єкти ефектів створюються лише один раз і повторно використовуються для обробки зображень.

Масштабованість: легко додати нові ефекти без зміни основного коду.

Продуктивність: зменшується кількість об'єктів у пам'яті, що підвищує ефективність роботи системи.

Патерн flyweight дозволив оптимізувати архітектуру графічного редактора, зробивши її ефективною з точки зору використання пам'яті та масштабованою для подальшого розширення.

Діаграма класів для паттерну Flyweight

Діаграма класів, яка реалізовує патерн flyweight зображена на рисунку 9:

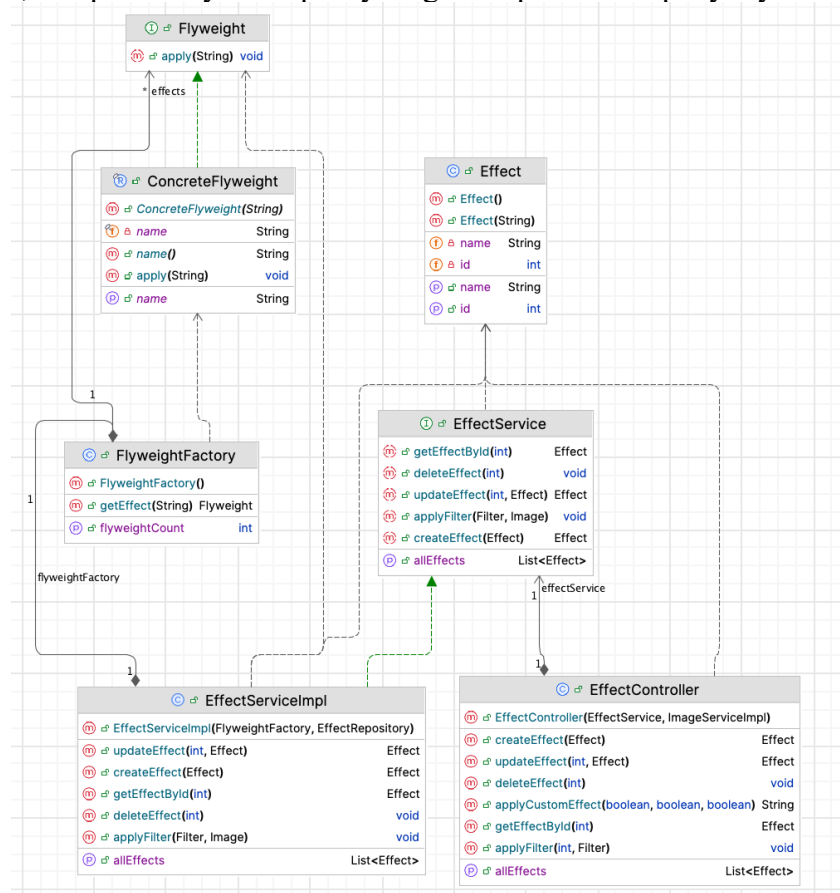


Рис.9 Діаграма класів для паттерну Flyweight

Ця діаграма класів представляє реалізацію паттерну Flyweight для оптимізації використання ресурсів при роботі з ефектами.

Основні компоненти:

1. Flyweight (Інтерфейс)

Визначає базову структуру для спільно використовуваних об'єктів (ефектів).

Методи:

- apply(String) — застосовує ефект до певного елементу.

2. ConcreteFlyweight (Клас)

Конкретна реалізація інтерфейсу Flyweight.

Зберігає стан, спільний для всіх екземплярів.

Методи:

- Конструктор ConcreteFlyweight(String) — створює об'єкт ефекту.
- name() — повертає ім'я ефекту.
- apply(String) — застосовує ефект.

3. FlyweightFactory (Клас)

Відповідає за створення та управління екземплярами 'Flyweight'.

Забезпечує спільне використання вже створених ефектів для економії пам'яті.

Методи:

- `getEffect(String)` — повертає існуючий або створює новий екземпляр ефекту.
- `flyweightCount` — показує кількість створених ефектів.

4. `Effect` (Клас)

Представляє об'єкт ефекту із властивостями:

- `id` — ідентифікатор ефекту.
- `name` — назва ефекту.

5. `EffectService` (Інтерфейс)

Визначає бізнес-логіку для роботи з ефектами.

6. `EffectServiceImpl` (Клас)

Реалізація інтерфейсу `EffectService`.

Координує роботу між `FlyweightFactory`, сховищем (`EffectRepository`) та бізнес-логікою.

7. `EffectController` (Клас)

Контролер, що забезпечує API для роботи з ефектами.

Зв'язки:

1. `FlyweightFactory` створює та керує об'єктами `Flyweight`, які представляють загальні ефекти.
 2. `EffectServiceImpl` використовує `FlyweightFactory` для роботи з ефектами та забезпечення їх повторного використання.
 3. `EffectController` надає API для доступу до сервісу ефектів (`EffectServiceImpl`).
- Ця реалізація дозволяє зменшити кількість об'єктів у пам'яті за допомогою спільного використання стану, зберігаючи гнучкість у роботі з ефектами.

Проблема, яку допоміг вирішити шаблон `Flyweight`

Шаблон `flyweight` у системі графічного редактора вирішив кілька важливих проблем, пов'язаних з оптимізацією роботи з ресурсами при обробці великої кількості об'єктів, таких як ефекти для зображень.

1. Оптимізація використання пам'яті

Графічний редактор дозволяє працювати з різними ефектами, які можуть повторно застосовуватись до багатьох зображень. Без використання шаблону `flyweight` для кожного ефекту створювався б новий об'єкт, що призвело б до значного перевантаження пам'яті. Ефекти (наприклад, "Grayscale", "Sepia") зберігаються в єдиному екземплярі, незалежно від того, скільки разів вони використовуються.

Використовується пул об'єктів (`FlyweightFactory`), який запобігає дублюванню однакових об'єктів.

2. Зменшення дублювання коду

Без шаблону `flyweight` потрібно було б створювати окремий екземпляр класу для кожного нового ефекту, навіть якщо ефект ідентичний. Це б призвело до дублювання коду та складності в підтримці.

Усі ефекти реалізуються через спільний інтерфейс `Flyweight`.

Створення нових ефектів автоматизовано за допомогою фабрики (`FlyweightFactory`), що дозволяє уникнути дублювання.

3. Управління великою кількістю об'єктів

При роботі з великою кількістю зображень і ефектів редактор стикається з проблемою управління численними екземплярами класів, що може вплинути на продуктивність.

Використовується єдиний екземпляр об'єкта для кожного унікального ефекту.

Навіть якщо тисячі зображень використовують один і той самий ефект, об'єкт створюється лише один раз.

4. Масштабованість та легкість розширення

Без шаблону flyweight додавання нового ефекту вимагало б суттєвих змін у коді, зокрема створення нових класів або екземплярів.

Нові ефекти можна легко додавати, розширюючи фабрику або реалізуючи нові класи ConcreteFlyweight.

Фабрика автоматично забезпечує доступ до нових ефектів.

5. Зменшення витрат на обробку

Для застосування ефектів важливо мінімізувати накладні витрати. Без шаблону flyweight витрати на створення нових об'єктів для кожного ефекту були б значними.

Використовується один екземпляр ефекту, що значно скорочує витрати на створення та управління об'єктами.

Шаблон flyweight допоміг створити ефективну та масштабовану систему для роботи з ефектами у графічному редакторі. Завдяки йому вдалося мінімізувати споживання пам'яті, уникнути дублювання коду та забезпечити високу продуктивність навіть при роботі з великими наборами зображень.

Переваги застосування патерну Flyweight

1. Ефективне використання пам'яті

Завдяки Flyweight вдалося уникнути створення численних однакових об'єктів для кожного ефекту. Це дозволило значно зменшити обсяг пам'яті, необхідний для зберігання об'єктів, таких як "Grayscale" чи "Sepia".

2. Оптимізація продуктивності

Патерн зменшує час, необхідний для створення нових об'єктів. Наприклад, замість того, щоб щоразу створювати новий екземпляр ефекту, використовується вже існуючий через фабрику.

3. Спрощення структури проєкту

Flyweight дозволив уникнути створення дублюючих класів для кожного ефекту або його варіації. Наприклад, не потрібно створювати окремий клас для кожної комбінації ефекту та даних.

4. Масштабованість

Легко додавати нові ефекти шляхом створення нових екземплярів у фабриці. Наприклад, новий ефект "Invert" можна додати без зміни існуючих класів.

5. Зменшення дублювання коду

Логіка створення та використання ефектів уніфікована за допомогою фабрики FlyweightFactory. Це дозволяє уникати дублювання при додаванні або застосуванні ефектів.

6. Гнучкість у налаштуванні

Flyweight дозволяє використовувати ті самі ефекти в різних контекстах, передаючи лише специфічні дані під час виконання. Наприклад, ефект "Grayscale" може бути застосований з різними параметрами до кількох зображень.

7. Поліпшення тестування

Оскільки об'єкти Flyweight є незалежними, їх можна тестувати окремо від інших компонентів. Наприклад, можна перевірити, чи правильно застосовується конкретний ефект "Sepia", не тестуючи весь редактор.

8. Простота інтеграції нових функцій

Додавання нових ефектів потребує лише розширення фабрики або створення нового класу ConcreteFlyweight. Наприклад, для додавання ефекту "Blur" достатньо реалізувати новий Flyweight.

9. Зниження складності

Використання пулу об'єктів замість створення нових дозволило спростити архітектуру системи. Це зробило код більш зрозумілим і легким у підтримці.

Завдяки патерну Flyweight графічний редактор став більш оптимізованим і масштабованим. Він забезпечує ефективне використання ресурсів, спрощує управління великою кількістю об'єктів і дозволяє легко інтегрувати новий функціонал.

Посилання на GitHub

Посилання на репозиторій: <https://github.com/Eliviraaaaaa/GraphicEditor.git>

Висновок

У даній роботі я реалізувала патерн проєктування Flyweight для графічного редактора, що дозволило оптимізувати використання пам'яті та підвищити продуктивність системи.

Завдяки цьому шаблону вдалося уникнути дублювання однакових об'єктів (наприклад, ефектів зображень), створивши пул спільно використовуваних екземплярів.

Застосування Flyweight допомогло знизити споживання ресурсів, особливо при роботі з великою кількістю ефектів, які повторюються. Це забезпечило масштабованість системи, дозволивши легко додавати нові ефекти або функціонал без змін у вже існуючому коді.

Архітектура стала більш гнучкою, зручною для підтримки й розширення, а також сприяла зменшенню складності проєкту.

Завдяки Flyweight графічний редактор став ефективним, ресурсозберігаючим і простим у розширенні, що відповідає сучасним вимогам до високопродуктивних систем.